

**FUNDAÇÃO DE ASSISTÊNCIA E EDUCAÇÃO
CENTRO UNIVERSITÁRIO ESPÍRITO-SANTENSE
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO**

**BERNARDO ANTÔNIO MERLO SOARES
RAFAEL BARCELOS DE AQUINO MOURA**

**VALIDAÇÃO DE DOCUMENTAÇÃO DE SISTEMAS
UTILIZANDO INTELIGÊNCIA ARTIFICIAL**

**VITÓRIA
2026**

BERNARDO ANTÔNIO MERLO SOARES
RAFAEL BARCELOS DE AQUINO MOURA

VALIDAÇÃO DE DOCUMENTAÇÃO DE SISTEMAS
UTILIZANDO INTELIGÊNCIA ARTIFICIAL

Projeto de Pesquisa do Trabalho de Conclusão de Curso de Graduação em Ciência da Computação apresentado ao Centro Universitário Espírito- Santense, sob orientação do professor Fernando Antônio Marques Filho.

VITÓRIA
2026

SUMÁRIO

1 INTRODUÇÃO.....	3
1.1 O PROBLEMA.....	3
1.2 FORMULAÇÃO DO PROBLEMA.....	4
1.3 HIPÓTESE.....	4
1.4 OBJETIVOS.....	4
1.4.1 Objetivo Geral.....	4
1.4.2 Objetivos Específicos.....	4
1.5 JUSTIFICATIVA.....	5
2 REFERENCIAL TEÓRICO.....	6
2.1 INTELIGÊNCIA ARTIFICIAL E PROCESSAMENTO DE LINGUAGEM NATURAL.....	6
2.2 DESAFIOS DA DOCUMENTAÇÃO DE SOFTWARE E A LGPD.....	6
2.3 GERAÇÃO AUMENTADA POR RECUPERAÇÃO (RAG) EM AMBIENTES ON-PREMISE.....	7
2.4 INTEGRAÇÃO AO FLUXO DE VERSIONAMENTO E VALIDAÇÃO SEMÂNTICA. 8	
2.4.1 Representação Vetorial e Similaridade de Cosseno.....	8
2.4.2 Documentação como Código (Doc-as-Code).....	8
3 METODOLOGIA.....	9
4 CRONOGRAMA.....	11
REFERÊNCIAS.....	12

1 INTRODUÇÃO

1.1 O PROBLEMA

Atualmente, o desenvolvimento de softwares empresariais envolve repositórios extensos e em constante evolução. Conforme o código-fonte é atualizado, a documentação atrelada a ele frequentemente torna-se obsoleta, incompleta ou inconsistente. Esse descompasso entre o que o sistema de fato executa e o que está documentado prejudica a manutenção, a integração de novos desenvolvedores e o entendimento global da arquitetura. A validação manual dessa correspondência é uma tarefa complexa, custosa e suscetível a falhas humanas.

Ferramentas como LLMs (Large Language Models) são úteis para analisar e comparar grandes volumes de texto e código. Contudo, devido a políticas empresariais de segurança da informação e diretrizes da LGPD (Lei Geral de Proteção de Dados), o envio de código-fonte proprietário e documentações internas para fornecedores externos (em nuvem) não é viável. A alternativa é utilizar LLMs executadas localmente (on-Premise) na infraestrutura da empresa. No entanto, estes modelos costumam ter menor porte e janelas de contexto reduzidas, tornando impossível a leitura e comparação simultânea de repositórios inteiros e documentações extensas sem perda de coerência e precisão.

A técnica Retrieval-Augmented Generation (RAG) surge para contornar essa restrição técnica. O RAG atua como um intermediário inteligente, buscando em uma base vetorial apenas os trechos de código e os parágrafos de documentação mais correlatos à validação exigida no momento, entregando-os juntos para a LLM processar.

Diante disso, identifica-se uma lacuna tecnológica: a falta de uma solução computacional segura e local (on-premise) capaz de cruzar dados de código-fonte com documentações técnicas de forma inteligente, para validar e apontar inconsistências entre eles sem extrapolar os limites de contexto das IAs.

1.2 FORMULAÇÃO DO PROBLEMA

De que forma a integração de modelos de linguagem locais (on-premise) com a técnica RAG pode viabilizar a validação automatizada e a detecção de inconsistências entre a documentação de sistemas e seus respectivos códigos-fonte, sem externalizar dados e respeitando os limites de contexto da IA?

1.3 HIPÓTESE

A técnica RAG supera as limitações de contexto de LLMs on-premise ao recuperar apenas os trechos de código e as seções de documentação diretamente relacionados, permitindo que a IA realize uma análise comparativa precisa para validar o sistema e identificar divergências sem a necessidade de externalização das informações.

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Desenvolver uma ferramenta baseada em LLMs *on-premise* e na técnica RAG capaz de validar automaticamente a consistência entre o código-fonte e a documentação técnica de sistemas, apontando divergências sem a externalização dos dados.

1.4.2 Objetivos Específicos

- Investigar as limitações de janela de contexto em LLMs de código aberto (Open Source) executadas em ambiente on-premise.
- Analisar a aplicação da técnica RAG no processamento cruzado de código-fonte e textos de documentação .
- Definir e testar estratégias de indexação vetorial e recuperação eficientes para parear com precisão trechos de código com suas respectivas regras de negócio e documentações .
- Modelar uma solução computacional que integre o RAG à LLM local focada em análise comparativa e validação .
- Construir a solução proposta.

- Avaliar a precisão da ferramenta na detecção de "falsos positivos" e na identificação real de divergências entre código e documentação.
- Validar a hipótese de que a utilização do RAG resolve a restrição de contexto para a validação de sistemas em ambientes isolados

1.5 JUSTIFICATIVA

A evolução ágil e contínua de softwares frequentemente resulta em documentações defasadas, o que aumenta a dívida técnica de um projeto e prejudica a confiabilidade das informações do sistema. Validar manualmente se a documentação reflete o código real exige um esforço humano muitas vezes proibitivo no dia a dia das equipes de engenharia.

Embora o uso de Inteligência Artificial seja a resposta natural para automatizar a validação e auditoria entre código e texto, esbarra-se no sigilo industrial e na LGPD, que proíbem o uso de ferramentas de IA públicas. A execução local de modelos de linguagem resolve o problema da privacidade, mas falha na limitação tecnológica da janela de contexto ao tentar analisar milhares de linhas de código e páginas de documentação de uma só vez.

Nesse cenário, o uso integrado da técnica RAG justifica-se técnica e comercialmente. Ao fatiar, indexar e fornecer à LLM apenas as contrapartes exatas de código e texto necessárias para uma validação pontual, viabiliza-se uma solução segura, escalável e de alto desempenho, assegurando a qualidade contínua da engenharia de software dentro dos limites de segurança das empresas.

2 REFERENCIAL TEÓRICO

2.1 INTELIGÊNCIA ARTIFICIAL E PROCESSAMENTO DE LINGUAGEM NATURAL

A Inteligência Artificial (IA) tem se consolidado como uma base tecnológica essencial na ciência da computação para a resolução de problemas complexos e repetitivos. Segundo Russell e Norvig (2020), a IA consiste no estudo e no design de agentes inteligentes, que têm a capacidade de perceber o seu ambiente e tomar ações visando maximizar suas chances de sucesso em tarefas específicas.

Essa capacidade de adaptação e processamento lógico torna a IA uma candidata ideal para analisar e otimizar grandes volumes de dados dentro da engenharia de software contemporânea.

Dentro do vasto campo da inteligência artificial, destaca-se especificamente o Processamento de Linguagem Natural (PLN), que aproxima a lógica computacional da semântica humana. Jurafsky e Martin (2026) explicam que o PLN é a subárea que permite que as máquinas compreendam, interpretem e gerem textos em linguagem humana de forma coerente.

A aplicação dessa tecnologia em ambientes corporativos é o que torna possível que algoritmos leiam códigos-fonte e documentações textuais de forma simultânea, extraindo o real significado das regras de negócio para facilitar auditorias automatizadas.

2.2 DESAFIOS DA DOCUMENTAÇÃO DE SOFTWARE E A LGPD

A manutenção da qualidade em projetos de software exige documentações precisas e atualizadas, porém, a evolução ágil e constante do código frequentemente gera severas defasagens. Conforme o código-fonte sofre atualizações contínuas nos repositórios empresariais, a documentação atrelada a ele torna-se obsoleta, incompleta ou inconsistente.

Esse descompasso entre a execução real da aplicação e o que está formalmente documentado eleva expressivamente o custo de manutenção, dificulta a integração de novas equipes e aumenta a dívida técnica da organização (Ballou, 2006).

Para solucionar esse problema de validação documental por meio de inteligência artificial, a indústria depara-se com um desafio crítico envolvendo a privacidade dos dados e as diretrizes da Lei Geral de Proteção de Dados (LGPD). O envio de códigos proprietários e infraestruturas internas para fornecedores de serviços em nuvem externa é muitas vezes proibido por políticas de sigilo.

Devido a isso, exige-se a adoção de grandes modelos de linguagem (LLMs) executados localmente, também conhecidos como modelos on-premise, garantindo que as informações estratégicas jamais deixem o perímetro seguro da organização.

2.3 GERAÇÃO AUMENTADA POR RECUPERAÇÃO (RAG) EM AMBIENTES ON-PREMISE

Apesar de resolverem o problema da segurança da informação, os modelos locais de inteligência artificial lidam com restrições severas de processamento e de janela de contexto. Devido ao menor porte dessas infraestruturas on-premise, torna-se tecnicamente inviável fornecer à IA a leitura e a comparação simultânea de repositórios inteiros de código e documentações extensas sem que haja perda de coerência.

Quando o limite de processamento de entrada de dados da IA é atingido, o modelo perde a capacidade de correlacionar as regras de negócio corretamente, inviabilizando a auditoria.

Para contornar as limitações físicas de contexto das LLMs locais, a técnica Retrieval-Augmented Generation (RAG) tem se mostrado a abordagem metodológica e técnica mais promissora. O RAG atua como um mecanismo intermediário e seletivo, que busca em uma base de dados vetorial apenas os trechos de código específicos e os parágrafos de documentação que estão intimamente correlacionados à validação pontual exigida naquele instante.

Ao fatiar e limitar os dados entregues à IA apenas ao que é estritamente necessário para a análise comparativa, o RAG permite superar a barreira da janela de contexto do hardware local, consolidando uma validação sistêmica de alta precisão e total conformidade com a privacidade corporativa.

2.4 INTEGRAÇÃO AO FLUXO DE VERSIONAMENTO E VALIDAÇÃO SEMÂNTICA

A integração da auditoria documental ao sistema de controle de versões, como o Git, permite que a validação ocorra de forma contínua e incremental. Segundo Chacon e Straub (2014), o uso de *Git Hooks* (ganchos de eventos) possibilita a execução de scripts automatizados em momentos críticos, como o *pre-commit* ou *pre-push*.

Nesse contexto, a ferramenta proposta atua analisando o git diff — a diferença entre a versão atual e a anterior do código. Em vez de processar todo o repositório, a técnica RAG utiliza as linhas modificadas como consulta para localizar, na base vetorial, os trechos correspondentes na documentação técnica.

2.4.1 Representação Vetorial e Similaridade de Cosseno

Para que a IA compare ativos de naturezas distintas (código e texto), ambos devem ser convertidos em *embeddings*, que são representações numéricas em um espaço vetorial de alta dimensionalidade. A consistência é verificada através da Similaridade de Cosseno, uma métrica que mede o cosseno do ângulo entre dois vetores. Quanto mais próximo de 1 o valor do cosseno, maior é a afinidade semântica entre o código e a descrição textual, permitindo que o sistema identifique divergências mesmo quando não há coincidência exata de palavras-chave.

2.4.2 Documentação como Código (Doc-as-Code)

A abordagem proposta alinha-se à filosofia *Doc-as-Code*, onde a documentação é tratada com o mesmo rigor que o software. Ao automatizar a verificação de integridade no momento do *commit*, estabelece-se um "portão de qualidade" (*quality gate*). Isso impede o acúmulo de dívida técnica, garantindo que qualquer evolução na lógica de negócio reflita imediatamente em uma atualização dos documentos de suporte, mantendo a "fonte da verdade" do projeto sempre íntegra.

3 METODOLOGIA

Esta pesquisa caracteriza-se como um estudo de natureza aplicada e exploratória, com abordagem mista (qualitativa e quantitativa). O objetivo central é desenvolver e validar uma ferramenta computacional que utiliza modelos de linguagem locais (on-premise) aliados à técnica RAG. Para garantir a reprodutibilidade e a clareza metodológica, as etapas de execução da pesquisa foram divididas da seguinte forma:

1. Coleta de Dados: A primeira etapa consistirá na seleção de um conjunto de dados (dataset) para servir de ambiente de teste. Serão coletados repositórios de código-fonte aberto (via plataformas como GitHub) que possuam documentações técnicas associadas. O critério de escolha priorizará repositórios que simulem a complexidade de um ambiente corporativo real. Além disso, serão introduzidas intencionalmente divergências ("falhas semânticas" e lógicas) entre o código e a documentação nesses repositórios para testar a capacidade de detecção da ferramenta.
2. Instrumentos e Tecnologias: Para o desenvolvimento da solução, a pesquisa utilizará um Modelo de Linguagem de Grande Escala (LLM) de código aberto (como Llama ou Mistral) executado em ambiente local (on-premise), assegurando a premissa de não externalização dos dados. Como instrumento de indexação, será configurado um banco de dados vetorial, responsável por armazenar os embeddings (representações matemáticas) gerados a partir da leitura dos códigos e das documentações.
3. Desenvolvimento da Solução: O desenvolvimento do sistema seguirá a estruturação de um pipeline RAG. Serão desenvolvidos scripts automatizados (integrados aos hooks do Git) que, ao identificarem uma atualização no código, extrairão apenas as linhas modificadas. Essas linhas serão transformadas em consultas vetoriais para buscar, no banco de dados, os trechos exatos de documentação correspondentes. O sistema enviará então esse contexto fatiado e altamente correlacionado para a LLM local, instruindo-a a comparar e apontar inconsistências.

4. **Análise e Avaliação dos Dados:** A validação da eficácia da ferramenta desenvolvida será feita de forma quantitativa. O sistema processará o conjunto de dados coletado na primeira etapa e seus resultados serão categorizados e analisados com base nas seguintes métricas:
- **Precisão:** A capacidade do sistema de identificar corretamente as divergências inseridas.
 - **Falsos Positivos e Negativos:** A taxa de vezes em que a IA acusa um erro inexistente ou deixa de perceber uma falha real.
 - **Consumo de Contexto:** A verificação empírica de que a técnica RAG manteve as requisições dentro do limite da janela de contexto da LLM on-premise, sem perda de coerência.

4 CRONOGRAMA

Atividades	AGO	SET	OUT	NOV
Leitura da bibliografia	X			
Coleta de Dados (Seleção de repositórios e injeção de divergências)	X			
Configuração de Instrumentos (LLM on-premise e BD Vetorial)	X	X		
Desenvolvimento da Solução (Pipeline RAG e Git hooks)		X		
Análise e Avaliação dos Dados (Testes de precisão e contexto)			X	
Redação da 1ª versão do texto			X	
Revisão do texto		X	X	
Redação definitiva			X	X
Entrega do TCC				X
Defesa				X

REFERÊNCIAS

- BALLOU, R. H. **Gerenciamento da Cadeia de Suprimentos/Logística Empresarial**. Tradução Rubenich, R. 5. ed. Porto Alegre: Bookman, 2006.
- CHACON, Scott; STRAUB, Ben. **Pro Git. 2.** ed. New York: Apress, 2014.
- FOWLER, M. **Refactoring**: Improving the Design of Existing Code. Addison-Wesley Professional, 2018.
- GENTLE, A. **Docs Like Code. 2.** ed. Austin: Lulu.com, 2018.
- JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. 3. ed. (draft), 2026.
- MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge University Press, 2008.